MATERI KECERDASAN KOMPUTASIONAL

Algoritma Genetika: dari Konsep ke Implementasi

Studi kasus optimasi knapsack dengan Python

Contents

1 Pendahuluan **3**

1.1 Masalah optimasi . . . 3

1.2 Kapan GA berguna? . . . 3

2 Konsep dan Alur Dasar **3**

2.1 Contoh yang digunakan: memilih barang . . . 4

3 Komponen Algoritma Genetika **4**

3.1 Representasi dan populasi awal . . . 4

3.2 Fungsi tujuan, fitness, dan batasan . . . 4

3.3 Seleksi induk . . . 5

3.4 Crossover . . . 5

3.5 Mutasi . . . 6

3.6 Elitisme, penggantian, dan penghentian . . . 6

3.7 Parameter yang perlu ditentukan . . . 6

4 Studi Kasus: Memilih Barang dalam Tas **7**

4.1 Data dan model matematika . . . 7

4.2 Evaluasi satu populasi contoh . . . 7

5 Pseudocode dan Implementasi Python **7**

5.1 Pseudocode . . . 7

5.2 Program lengkap . . . 8

6 Penerapan Lain dan Evaluasi **9**

7 Ringkasan **10**

1 Pendahuluan

1.1 Masalah optimasi

Optimasi berarti mencari solusi terbaik menurut ukuran tertentu sambil memenuhi batasan masalah. Contohnya memilih barang dengan nilai tertinggi tanpa melampaui kapasitas tas, menyusun jadwal yang meminimalkan bentrokan, atau menentukan parameter model yang menghasilkan galat terendah.

Jika jumlah kemungkinan solusi sangat besar, mencoba semuanya satu per satu dapat menjadi mahal. **Algoritma genetika** (*genetic algorithm*, GA) adalah metode pencarian berbasis populasi yang meniru beberapa gagasan evolusi: solusi yang lebih baik cenderung dipilih, kemudian dikombinasikan dan diubah untuk membentuk solusi baru [1, 2]. GA merupakan **heuristik stokastik**: hasilnya bergantung pada rancangan masalah dan unsur acak, sehingga tidak otomatis menjamin optimum global.

1.2 Kapan GA berguna?

GA relevan ketika solusi dapat dikodekan sebagai kromosom, kualitasnya dapat dihitung, dan ruang pencarian terlalu besar atau rumit untuk ditelusuri langsung. Modul ini memakai contoh empat barang yang diperkenalkan pada Bagian 2.1. Untuk masalah sekecil itu, enumerasi lengkap lebih sederhana; GA dipakai agar alurnya mudah dipelajari.

2 Konsep dan Alur Dasar

Satu **individu** adalah satu calon solusi. **Kromosom** adalah representasinya, sedangkan **gen** adalah satu bagian dari representasi itu. Sekumpulan individu disebut **populasi**. Pada tiap **generasi**, algoritma mengevaluasi individu, memilih calon induk, menghasilkan keturunan melalui *crossover* dan mutasi, lalu membentuk populasi berikutnya.

Istilah	**Makna operasional**
Gen dan kromosom	Elemen keputusan dan susunan lengkapnya; misalnya bit untuk tiap barang.
Fenotipe	Solusi yang diartikan dari kromosom; misalnya daftar barang terpilih.
Fitness	Skor kualitas yang digunakan selama seleksi.
Populasi dan generasi	Kumpulan calon solusi dan satu putaran pembaruannya.
Induk dan keturunan	Individu terpilih dan solusi baru yang dihasilkan.

Alur umum:

1. Tentukan representasi solusi, fungsi fitness, batasan, dan parameter GA.
2. Bangkitkan populasi awal, lalu hitung fitness setiap individu.
3. Simpan beberapa individu terbaik jika memakai elitisme.
4. Pilih induk, lakukan crossover dan mutasi, lalu evaluasi keturunan.
5. Ulangi sampai kriteria berhenti terpenuhi; kembalikan solusi terbaik yang ditemukan.

2.1 Contoh yang digunakan: memilih barang

Agar setiap komponen GA dapat dilihat pada masalah yang sama, bagian berikut memakai contoh *knapsack*: memilih barang untuk dimasukkan ke tas. Setiap barang boleh dipilih satu kali. Tujuannya adalah memperoleh nilai total setinggi mungkin dengan bobot total paling banyak 8 kg.

Barang	Bobot (kg)	Nilai
A	2	6
B	3	5
C	4	8
D	5	9

Keputusan memilih barang ditulis sebagai empat bit berurutan A–B–C–D: angka 1 berarti barang dipilih, dan 0 berarti tidak. Misalnya, `1001` berarti memilih A dan D. Bagian 3 memakai contoh ini untuk menjelaskan tiap komponen; Bagian 4 menyajikan model matematika dan perhitungan fitness-nya secara lengkap.

3 Komponen Algoritma Genetika

3.1 Representasi dan populasi awal

Representasi harus sesuai dengan jenis keputusan. Bit biner cocok untuk keputusan ya/tidak, bilangan riil untuk parameter kontinu, dan permutasi untuk urutan kunjungan atau tugas. Operator genetika juga harus cocok dengan representasi: crossover satu titik pada permutasi dapat menimbulkan elemen ganda, sehingga masalah rute memerlukan operator yang mempertahankan permutasi [2, 4].

Untuk empat barang A, B, C, D, kromosom `1001` berarti memilih A dan D. Populasi awal biasanya dibangkitkan acak agar tersedia beberapa titik awal pencarian. Ukuran populasi yang lebih besar menambah keragaman sekaligus jumlah evaluasi.

3.2 Fungsi tujuan, fitness, dan batasan

Fungsi tujuan (*objective function*) menjelaskan hasil yang ingin dicapai. Pada contoh tas, tujuannya adalah memperoleh *nilai barang setinggi mungkin*. **Fitness** adalah angka yang dipakai GA untuk menilai setiap calon solusi. Semakin besar fitness, semakin baik calon solusi tersebut menurut aturan penilaian yang ditetapkan. Jika tujuan memang memaksimalkan nilai dan tidak ada masalah batasan, fitness dapat langsung diisi dengan nilai barang.

Batasan menentukan solusi yang boleh digunakan. Tas pada contoh ini hanya dapat membawa 8 kg. Kromosom `1001` memilih A dan D: bobotnya 7 kg, nilainya 15, sehingga fitness-nya 15. Kromosom `1111` memilih semua barang: nilainya 28, tetapi bobotnya 14 kg. Karena melewati kapasitas, contoh program memberinya fitness 0. Dengan demikian, nilai barang yang tinggi belum tentu berarti solusi tersebut baik atau dapat dipakai.

Pemberian fitness 0 kepada solusi yang melanggar batasan disebut **penalti**. Cara lain adalah memperbaiki solusi yang melanggar atau merancang operator agar hanya menghasilkan solusi yang memenuhi batasan [5]. Untuk masalah yang tujuannya *meminimalkan* sesuatu, seperti keterlambatan, skor dapat dibalik agar nilai fitness yang lebih besar tetap berarti lebih baik.

Sebagai contoh, jika keterlambatan tidak negatif, $F = 1/(1 + \text{keterlambatan})$ memberi skor lebih tinggi pada keterlambatan yang lebih kecil.

3.3 Seleksi induk

Seleksi menentukan individu yang akan menjadi induk. Pada **seleksi turnamen**, GA mengambil beberapa individu secara acak dari populasi, lalu memilih yang fitness-nya paling tinggi. Contoh program memakai dua peserta dalam setiap turnamen. Ilustrasi berikut menampilkan dua turnamen untuk memilih dua induk; angka dalam kurung adalah fitness [4, 6].

Populasi: `1001` (15), `0110` (13), `1100` (11), `1111` (0)

Putaran	Dua peserta yang terambil	Induk terpilih
1	`1001` (15) vs. `0110` (13)	$\longrightarrow$ `1001` (15)
2	`0110` (13) vs. `1100` (11)	$\longrightarrow$ `0110` (13)

Pemilihan peserta pada ilustrasi hanyalah satu kemungkinan hasil acak. Turnamen berikutnya kembali mengambil peserta dari populasi yang sama, sehingga satu individu dapat terpilih menjadi induk lebih dari sekali. Metode seleksi lain mencakup seleksi proporsional fitness dan seleksi berdasarkan peringkat [3].

3.4 Crossover

Crossover membuat anak dengan menggabungkan gen dari dua induk. Pada *crossover satu titik*, cukup pilih satu batas di antara gen, kemudian tukar bagian setelah batas itu. Misalnya, gunakan dua induk terpilih pada ilustrasi sebelumnya. Urutan gen selalu A–B–C–D.

1. **Tentukan titik potong.** Potong setelah gen B. Bagian kiri berisi keputusan untuk A dan B; bagian kanan berisi keputusan untuk C dan D.
2. **Tukar bagian kanan.** Anak 1 mengambil bagian kiri induk 1 dan bagian kanan induk 2. Anak 2 mengambil bagian kiri induk 2 dan bagian kanan induk 1. Hasilnya:

	Kiri (A, B)	Kanan (C, D)	Kromosom lengkap
Induk 1	`10`	`01`	`1001`
Induk 2	`01`	`10`	`0110`
Anak 1	`10` dari induk 1	`10` dari induk 2	`1010`
Anak 2	`01` dari induk 2	`01` dari induk 1	`0101`

3. **Nilai kembali kedua anak.** `1010` berarti A dan C (bobot 6 kg, nilai 14); `0101` berarti B dan D (bobot 8 kg, nilai 14). Anak belum tentu lebih baik daripada induknya.

Peluang crossover $p_c = 0{,}8$ pada contoh program berarti setiap pasangan induk memiliki peluang 80% untuk mengalami pertukaran. Jika tidak terjadi crossover, kedua anak merupakan salinan induknya.

3.5 Mutasi

Mutasi mengubah satu atau beberapa gen secara acak. Pada kromosom biner, perubahan ini disebut *bit-flip*: 0 menjadi 1 atau 1 menjadi 0. Mutasi memberi kesempatan untuk mencoba kombinasi yang belum muncul melalui crossover [4].

Sebagai contoh, kromosom `1000` hanya memilih A (bobot 2 kg, nilai 6). Jika gen D berubah dari 0 menjadi 1, hasilnya adalah

	A	**B**	**C**	**D**		**Barang terpilih**
Sebelum mutasi	1	0	0	**0**	$\longrightarrow$	A
Sesudah mutasi	1	0	0	**1**	$\longrightarrow$	A dan D

Kromosom baru `1001` memilih A dan D dengan bobot 7 kg dan nilai 15. Contoh ini menunjukkan mutasi yang memperbaiki fitness; mutasi juga dapat menurunkannya atau membuat solusi melanggar batasan, sehingga setiap hasil perlu dinilai kembali.

Dalam kode, peluang mutasi $p_m = 0{,}1$ diterapkan *pada setiap gen anak*, bukan sekali untuk seluruh kromosom. Karena itu, satu anak dapat mengalami nol, satu, atau beberapa perubahan gen. Peluang yang terlalu tinggi dapat mengganggu solusi baik; peluang yang terlalu rendah dapat mengurangi variasi populasi.

3.6 Elitisme, penggantian, dan penghentian

Elitisme menyalin sejumlah individu terbaik tanpa perubahan ke generasi berikutnya. Karena individu terbaik dipertahankan, fitness terbaik pada populasi tidak turun dari satu generasi ke generasi berikutnya. Sisa tempat diisi keturunan baru. Kriteria berhenti dapat berupa batas generasi, batas evaluasi, target fitness, atau tidak adanya perbaikan selama beberapa generasi. Batas generasi membuat biaya komputasi mudah diperkirakan; ia tidak membuktikan solusi sudah optimal.

3.7 Parameter yang perlu ditentukan

Parameter	**Peran**	**Contoh kode**
Ukuran populasi N	Banyak calon solusi per generasi	20
Jumlah generasi G	Banyak putaran pembaruan	30
Peluang crossover p_c	Peluang pertukaran pada sepasang induk	0,8
Peluang mutasi p_m	Peluang perubahan pada setiap gen anak	0,1
Jumlah elit E	Individu terbaik yang disalin utuh	2

Nilai pada tabel adalah pengaturan contoh untuk masalah empat gen, bukan aturan baku. Biaya evaluasi kurang lebih bertambah bersama N dan G; variasi hasil perlu diperiksa dengan beberapa `seed`.

4 Studi Kasus: Memilih Barang dalam Tas

4.1 Data dan model matematika

Gunakan data empat barang dan kapasitas tas dari Bagian 2.1. Bagian ini menuliskan tujuan dan batasannya secara matematis agar perhitungan fitness pada contoh sebelumnya dapat diperiksa.

Definisikan $x_i \in \{0, 1\}$ untuk menunjukkan apakah barang i dipilih. Masalahnya adalah

$$\max V(\mathbf{x}) = \sum_{i=1}^{4} v_i x_i, \qquad \text{dengan } \sum_{i=1}^{4} w_i x_i \leq 8.$$

Fitness yang dipakai dalam GA:

$$F(\mathbf{x}) = \begin{cases} V(\mathbf{x}), & \sum_i w_i x_i \leq 8, \\ 0, & \sum_i w_i x_i > 8. \end{cases}$$

4.2 Evaluasi satu populasi contoh

Kromosom	Barang	Bobot	Nilai	Fitness
`1001`	A, D	7	15	15
`0110`	B, C	7	13	13
`1100`	A, B	5	11	11
`1111`	A, B, C, D	14	28	0

Solusi `1111` memiliki nilai total tinggi tetapi melanggar kapasitas. Jika turnamen membandingkan `1001` dan `0110`, pemenangnya `1001`. Contoh crossover di atas menunjukkan bahwa dua induk itu dapat menghasilkan `1010` (A, C; bobot 6; nilai 14) dan `0101` (B, D; bobot 8; nilai 14). Mutasi dapat mengubah salah satu anak lagi; fitness selalu dihitung ulang sesudahnya.

Untuk data kecil ini, pemeriksaan seluruh $2^4 = 16$ kromosom menunjukkan nilai maksimum 15, dicapai oleh A dan D. Perhitungan lengkap ini dipakai sebagai pembanding hasil GA, bukan bagian dari proses GA.

5 Pseudocode dan Implementasi Python

5.1 Pseudocode

```
buat populasi acak berukuran N
evaluasi fitness setiap individu
ulangi sampai G generasi:
    salin E individu terbaik ke populasi baru
    selama populasi baru belum berukuran N:
        pilih dua induk dengan turnamen
        dengan peluang pc, lakukan crossover
        mutasikan setiap gen anak dengan peluang pm
        tambahkan anak ke populasi baru
    evaluasi populasi baru
kembalikan individu terbaik
```

5.2 Program lengkap

Simpan kode berikut sebagai `ga_knapsack.py` lalu jalankan `python3 ga_knapsack.py`. Program hanya memakai pustaka standar Python. `seed` membuat hasil contoh dapat diulang pada implementasi Python yang sama.

```
import random

# Urutan gen: A, B, C, D
NAMA = ("A", "B", "C", "D")
BOBOT = (2, 3, 4, 5)
NILAI = (6, 5, 8, 9)
KAPASITAS = 8

def ukur(kromosom):
    berat = sum(b * gen for b, gen in zip(BOBOT, kromosom))
    nilai = sum(v * gen for v, gen in zip(NILAI, kromosom))
    return berat, nilai

def fitness(kromosom):
    berat, nilai = ukur(kromosom)
    return nilai if berat <= KAPASITAS else 0

def individu_acak(rng):
    return [rng.randrange(2) for _ in NAMA]

def turnamen(populasi, rng, ukuran=2):
    peserta = rng.sample(populasi, ukuran)
    return max(peserta, key=fitness)[:]

def crossover(induk1, induk2, rng, peluang):
    if rng.random() >= peluang:
        return induk1[:], induk2[:]
    titik = rng.randrange(1, len(induk1))
    anak1 = induk1[:titik] + induk2[titik:]
    anak2 = induk2[:titik] + induk1[titik:]
    return anak1, anak2

def mutasi(kromosom, rng, peluang):
    for i in range(len(kromosom)):
        if rng.random() < peluang:
            kromosom[i] = 1 - kromosom[i]
    return kromosom

def algoritma_genetika(seed=4, jumlah=20, generasi=30,
                       peluang_crossover=0.8, peluang_mutasi=0.1,
                       jumlah_elit=2):
    if not (2 <= jumlah and 0 <= jumlah_elit < jumlah):
        raise ValueError("Ukuran populasi/elit tidak valid")
    if not (0 <= peluang_crossover <= 1 and
            0 <= peluang_mutasi <= 1):
        raise ValueError("Peluang harus antara 0 dan 1")

    rng = random.Random(seed)
    populasi = [individu_acak(rng) for _ in range(jumlah)]
    riwayat = [max(map(fitness, populasi))]

    for _ in range(generasi):
        urut = sorted(populasi, key=fitness, reverse=True)
        baru = [individu[:] for individu in urut[:jumlah_elit]]
        while len(baru) < jumlah:
            induk1 = turnamen(populasi, rng)
            induk2 = turnamen(populasi, rng)
```

```
            anak1, anak2 = crossover(induk1, induk2, rng,
                                     peluang_crossover)
            for anak in (anak1, anak2):
                if len(baru) < jumlah:
                    baru.append(mutasi(anak, rng, peluang_mutasi))
        populasi = baru
        riwayat.append(max(map(fitness, populasi)))

    terbaik = max(populasi, key=fitness)
    return terbaik, riwayat

if __name__ == "__main__":
    terbaik, riwayat = algoritma_genetika()
    berat, nilai = ukur(terbaik)
    barang = [nama for nama, gen in zip(NAMA, terbaik) if gen]
    print("Kromosom:", "".join(map(str, terbaik)))
    print("Barang:", ", ".join(barang) or "(tidak ada)")
    print("Bobot:", berat, "kg")
    print("Nilai:", nilai)
    print("Fitness awal/akhir:", riwayat[0], "/", riwayat[-1])
```

Penjelasan alur kode. `ukur` menerjemahkan kromosom menjadi bobot dan nilai. `fitness` memberi nol pada solusi yang melanggar kapasitas. `turnamen` mengambil dua kandidat dari populasi dan menyalin pemenangnya. `crossover` dan `mutasi` membentuk variasi. Dua elit disalin langsung sebelum tempat lainnya diisi anak. `riwayat` menyimpan fitness terbaik pada populasi awal dan setelah tiap generasi.

Output untuk parameter bawaan:

```
Kromosom: 1001
Barang: A, D
Bobot: 7 kg
Nilai: 15
Fitness awal/akhir: 14 / 15
```

Pada `seed=4`, fitness terbaik meningkat dari 14 menjadi 15. Satu percobaan ini *bukan* bukti bahwa GA selalu menemukan optimum. Pada kasus ini, optimum 15 diketahui melalui enumerasi 16 kemungkinan; untuk masalah besar, pembuktian optimum memerlukan metode atau batas tambahan.

6 Penerapan Lain dan Evaluasi

Bidang	Representasi calon solusi	Contoh fitness
Penjadwalan	Slot waktu setiap tugas	Negatif penalti konflik dan keterlambatan.
Pemilihan fitur	Satu bit per fitur: dipakai atau tidak	Akurasi validasi dengan penalti jumlah fitur.
Penyetelan parameter	Vektor bilangan riil	Kinerja validasi model pada data yang terpisah.
Perutean	Permutasi urutan lokasi	Negatif jarak tempuh atau fungsi biaya.

Cara mengevaluasi GA. Jalankan beberapa `seed` dan laporkan nilai terbaik, rata-rata, penyebaran hasil, serta jumlah evaluasi fitness atau waktu komputasi. Bandingkan dengan metode

sederhana yang sesuai, misalnya enumerasi untuk data kecil atau pencarian acak dengan anggaran evaluasi sama. Periksa kelayakan solusi akhir secara terpisah. Untuk penerapan nyata, pilih parameter berdasarkan pengujian, bukan menganggap nilai p_c atau p_m tertentu selalu terbaik.

Keterbatasan. Evaluasi fitness dapat mahal; representasi dan penalti yang buruk dapat mengarahkan pencarian ke solusi yang tidak berguna; populasi dapat kehilangan keragaman dan berhenti berkembang. GA tepat diperlakukan sebagai alat pencarian yang perlu diuji terhadap kebutuhan dan metode pembanding, bukan sebagai jaminan solusi optimal [1, 2].

7 Ringkasan

Algoritma genetika mengulang siklus **representasi** → **evaluasi** → **seleksi** → **crossover** → **mutasi** → **penggantian**. Kualitas solusi ditentukan terutama oleh kecocokan representasi, fungsi fitness, penanganan batasan, dan anggaran pencarian. Pada contoh knapsack, kromosom `1001` memilih A dan D dengan bobot 7 kg serta nilai 15.

Daftar Pustaka

[1] D. Whitley, *A Genetic Algorithm Tutorial*, Statistics and Computing, vol. 4, hlm. 65–85, 1994. https://www.cs.colostate.edu/~genitor/MiscPubs/tutorial.pdf.

[2] A. E. Eiben dan J. E. Smith, *Genetic Algorithms*, dalam Introduction to Evolutionary Computing, Springer, 2003. https://doi.org/10.1007/978-3-662-05094-1_3.

[3] D. E. Goldberg dan K. Deb, *A Comparative Analysis of Selection Schemes Used in Genetic Algorithms*, dalam Foundations of Genetic Algorithms, 1991, hlm. 69–93. https://doi.org/10.1016/B978-0-08-050684-5.50008-2.

[4] DEAP, *Evolutionary Tools*, dokumentasi operator seleksi, crossover, dan mutasi. https://deap.readthedocs.io/en/stable/api/tools.html.

[5] DEAP, *Constraint Handling*, dokumentasi fungsi penalti. https://deap.readthedocs.io/en/stable/tutorials/advanced/constraints.html.

[6] MIT OpenCourseWare, *ESD.77 Lecture 11: Genetic Algorithms I*, hlm. 25, ilustrasi seleksi turnamen. Materi kuliah MIT OpenCourseWare (PDF).